Client-Side Lossless Decoding for Web-Based PACS: A 16-Bit-Native Codec for Real-Time DICOM Viewing

Kuldeep Singh

***Abstract*—Web-based Picture Archiving and Communication Systems (PACS) viewers must decode multi-frame Digital Imaging and Communications in Medicine (DICOM) studies in the browser at interactive rates (about 25–60 frames per second for smooth scrolling and cine playback) while remaining strictly lossless at the scanner's native 10–16-bit depth. Archival codecs that optimize ratio alone do not automatically meet that constraint, and several high-ratio formats lack a practical browser path for 16-bit lossless data. This paper presents MIC (Medical Image Codec), a lossless pipeline built for that viewer workflow: spatial prediction, 16-bit run-length encoding (RLE), and large-alphabet Finite State Entropy (FSE, a table-driven form of asymmetric numeral systems), with a multi-state decoder and strip-parallel mode that keep client-side decode above cine frame rates. On a deliberately diverse systems-benchmark corpus of 39 DICOM images across eleven modalities, MIC reaches a geometric-mean compression ratio of 3.36× (about 10% smaller files than Delta + Zstandard on 34 of 39 images; within about 2% of High-Throughput JPEG 2000 and about 88% of JPEG-LS), is the fastest of four in-process decoders on all 39 images on ARM64 and 36 of 39 on AMD64, and ships a ∼20 KB pure-JavaScript decoder that sustains interactive multi-frame playback in the browser without server-side transcoding. JPEG XL is discussed as a ratio baseline but is excluded from the viewer comparison because no current browser decodes it losslessly at 16-bit depth.**

***Index Terms*—medical image compression, lossless compression, PACS, web viewer, DICOM, browser decoding, entropy coding, asymmetric numeral systems (ANS), finite state entropy (FSE), high-bit-depth imaging.**

I. Real-Time Viewing as the Design Constraint

WEB-BASED Picture Archiving and Communication Systems (PACS) viewers have become a primary path for clinical image review: a radiologist opens a study in the browser, scrolls a multi-slice computed tomography (CT) series or a digital breast tomosynthesis (DBT) stack, and expects each frame to appear without a perceptible stall. That interaction is not an archival workload. Archival storage optimizes primarily for file size written once and read rarely; a PACS viewer must continuously decode pixels at interactive rates while remaining strictly lossless, because diagnostic review requires that the reconstructed image match the scanner output bit for bit.

K. Singh is with Innovaccer Inc. E-mail: kuldeep.singh@innovaccer.com. ORCID: 0009-0004-8476-0118. Code: https://github.com/pappuks/medical-image-codec.

The practical bottleneck in that workflow, based on experience building clinical PACS viewer software, is sustaining roughly **25–60 frames per second (FPS)** of decoded 10–16-bit pixels for smooth scrolling and cine playback. A modern scanner produces large studies: a single tomosynthesis view can comprise 60 or more slices and over 600 MB uncompressed, and a full screening study can exceed 2 GB. Moving that volume over the network already motivates compression; displaying it interactively in the browser further requires that decompression be fast enough on the client that frame rate, not file size alone, is the binding constraint. A codec that saves a few percent in storage but cannot sustain cine rates degrades the viewing experience regardless of its archival efficiency.

Digital Imaging and Communications in Medicine (DICOM) [1] already supports several lossless formats, including JPEG 2000 [2], High-Throughput JPEG 2000 (HTJ2K) [3], JPEG-LS [5] (a lossless/near-lossless predictive JPEG standard), and a basic run-length scheme. Each makes a different trade-off among ratio, decode speed, and implementation complexity [22], [23]. JPEG-LS usually produces the smallest files. HTJ2K improves on classical JPEG 2000 decode speed. Neither was designed around the joint constraint this paper targets: **native 16-bit lossless decode, sustained interactive frame rate, and deployability as a small client-side decoder in a browser-based PACS**. Server-side transcoding can paper over format gaps, but it adds latency, cost, and an extra failure path; a purpose-built client-side path avoids that transcoding step.

A. Target workflow: client-side multi-frame viewing

The viewer path comprises three stages that must stay within the end-to-end frame budget:

1) **Fetch** a compressed frame (or strip-parallel blob) from object storage or a PACS gateway into the browser.
2) **Decode** the frame to native-depth pixels on the client (JavaScript or WebAssembly), without a server re-encoding step.
3) **Display** the frame in a canvas or WebGL viewport while the user scrolls or plays cine.

Under that model the codec succeeds only if decode time per frame leaves enough budget for fetch and display at 25–60 FPS. Large mammography and tomosynthesis frames are the stress cases; small magnetic resonance (MR) and positron

emission tomography (PET) frames are latency-sensitive for series scrubbing. Section VII-A reports in-browser cine measurements against this target; Section VI establishes the ratio and native throughput context.

B. Why 16-bit-native coding matters for that workflow

Medical pixels are commonly 10–16 bits wide, not the 8-bit samples assumed by most general-purpose entropy coders (Huffman, arithmetic coding, the LZ77 family, and the Finite State Entropy / asymmetric numeral systems stage inside Zstandard [14]). When an 8-bit coder is applied to 16-bit residuals, it has two poor options:

1) **Split each residual into a high byte and a low byte** and code them separately. This doubles the number of coding steps and discards the relationship between the two bytes.
2) **Cap the alphabet** and store rarer values as raw data, which wastes space.

In contrast to codecs that split 16-bit pixels into byte pairs, a 16-bit-native design treats each predicted residual as a single 16-bit symbol, so the entropy model can capture correlations between the high and low byte directly. For example, if the predicted value is 1020 and the true pixel is 1023, the residual is +3. Residuals near zero dominate smooth tissue; the high byte of a small signed residual is nearly deterministic given the low byte, yet a byte-oriented coder still spends bits on it. On the test images the mutual information between the two bytes ranges from 0.3 to 1.1 bits per pixel, which is an upper bound on the waste of byte-splitting. Section VI reports that MIC produces files 5–22% smaller than Delta + Zstandard coding the same residuals after byte-splitting; that measured advantage falls within the mutual-information ceiling and is consistent with the shared-information argument as the main source of the gain. For a web viewer, smaller files also reduce fetch time, so the same design choice that improves ratio helps the end-to-end frame budget.

C. What MIC is

MIC (Medical Image Codec) is a three-stage lossless pipeline built for high decode throughput on medical residuals: (i) a simple spatial predictor (average of the top and left neighbors), (ii) a 16-bit run-length encoder, and (iii) an FSE entropy coder extended to large alphabets (up to 65,535 symbols), plus a multi-state decoder and an optional strip-parallel mode called Parallel Image Compressed Strips (PICS). PICS splits an image into N horizontal strips, each with its own FSE table and independently decodable bitstream, so a worker pool can decompress strips concurrently without changing the ratio of a given configuration. A ~20 KB pure-JavaScript decoder and a WebAssembly build implement the same bitstream for client-side use.

D. Contributions

- A **viewer-oriented problem framing**: lossless compression judged by whether client-side decode can sustain interactive multi-frame rates in a web PACS, not by archival ratio alone (Sections I-A, VII-A).
- A **16-bit-native pipeline** (Delta + 16-bit RLE + extended FSE) that outperforms Delta + Zstandard on 34 of 39 images (about 10% smaller in geometric mean) while remaining simple enough for a small browser decoder.
- A **multi-state and strip-parallel decode path** that keeps single-threaded and multi-threaded throughput above cine targets on heavy studies, including tomosynthesis-sized frames.
- An **evaluation against HTJ2K, JPEG-LS, and Delta + Zstandard** under in-process methodology, with JPEG XL discussed as a ratio-capable but browser-infeasible baseline for 16-bit lossless viewing (Section II).
- A **purpose-built JavaScript/WebAssembly decoder** (~20 KB JS) that fetches and decodes `.mic` content client-side without server-side transcoding.

a) Why this combination is the contribution: Spatial prediction, run-length coding, and ANS/FSE are each well established, so the claim is not inventing those stages. The contribution is that pairing a 16-bit-native entropy stage with multi-state parallel ANS decoding, and shipping that path as a small browser decoder, removes a specific bottleneck: sustained real-time, browser-based decoding of DICOM images at native 16-bit depth. JPEG-LS's Golomb–Rice stage and HTJ2K's block coder retain per-symbol data-dependent branches that limit decode throughput; Zstandard's FSE is fixed at an 8-bit-oriented, 4,096-symbol alphabet. None of those pipelines simultaneously targets native 16-bit alphabet width, multi-state interleaved decode, and a purpose-built browser deployment for interactive PACS viewing. Section VI and Section VII-A show the combination meeting the frame-rate target that archival-ratio-focused codecs were not designed to guarantee.

b) Evaluation scope and statistical framing: Claims in this paper are about **decoder throughput and ratio on a deliberately diverse modality set**, following the systems-benchmark tradition (for example, Kodak's 24-image suite and CLIC validation sets of roughly 41–61 images), not clinical validation of diagnostic interchangeability on a population archive. Clinical-validation studies of lossless codecs often use thousands of images [23]; the 39-image, eleven-modality corpus here is chosen for breadth of pixel statistics and viewer stress cases, not for population inference. Larger multi-hospital validation is left to future work (Section VII).

c) Scope: Primary results cover lossless grayscale decode for single-frame and multi-frame viewing paths. Color whole-slide imaging, lossy modes, and progressive decoding are out of scope.

II. Related Work

The DICOM lossless codecs. JPEG 2000 [2] uses a wavelet transform plus a sophisticated block coder (EBCOT): strong ratios, but slow decoding. HTJ2K [3], [4] replaces the block coder with a faster one and decodes considerably quicker. JPEG-LS [5] uses a small predictor (Median Edge Detector) and Golomb-Rice coding; it is a widely deployed standard

and a strong ratio baseline. The basic DICOM run-length scheme compresses little because it works on raw bytes with no prediction step. Research codecs such as CALIC [6] and FLIF [7] compress well on natural images but are not DICOM standards and lack browser decoders.

JPEG XL. JPEG XL [8] is a general-purpose image format whose modular lossless mode also handles high-bit-depth data, and its absence from the quantitative comparison deserves an explicit explanation rather than a silent omission. In informal cross-checks using the reference `libjxl` encoder and decoder called in-process (effort level 7; the harness is included in the accompanying code as `ojph/jxl_comparison_test.go`), JPEG XL reaches a modestly better compression ratio than MIC on the same images, while MIC encodes and decodes markedly faster; a full quantitative comparison on the reference platforms of Section V, following the same in-process methodology used for HTJ2K and JPEG-LS, is left to future work (Section VII). The more important reason JPEG XL is not a baseline here is that it is not a feasible option for the browser-viewer workflow this paper targets: the only mature browser-side JPEG XL decoder available at the time of writing, `@jsquash/jxl`, returns an 8-bit RGBA `ImageData` object and so cannot represent lossless 16-bit grayscale medical pixels; no current browser can decode JPEG XL losslessly at 16-bit depth. The comparison is therefore not only a matter of ratio versus speed but of baseline feasibility for a client-side, 16-bit lossless viewer. Newer learned lossless codecs, such as L3C [9] and SReC [10], and other autoregressive or hierarchical range-coding approaches, can reach still better ratios, but depend on neural-network runtimes and decode far more slowly, which makes them impractical for real-time web viewing; they are excluded from the quantitative comparison for the same reason.

These are the codecs MIC is measured against for the viewer-relevant axes of ratio, native decode speed, and browser deployability. The goal is not to outperform archival codecs on every metric, but to show a residual-domain design that is fast enough for interactive web PACS while remaining competitive on ratio and simple enough to ship as a small client-side decoder.

Entropy coding background. Asymmetric Numeral Systems (ANS) [11], [12] is a family of entropy coders that replaces arithmetic coding's interval math with simple integer state updates. FSE is a fast, table-driven version of ANS [13] made popular by Zstandard [14]. Coding directly over a large, multi-symbol alphabet rather than a byte stream is not itself new; for example, Mahapatra and Singh built a multi-alphabet arithmetic coder in FPGA hardware [15]. The limitation is that the widely deployed software implementations are tuned for byte streams (Zstandard caps FSE at 4,096 symbols).

Parallel ANS. Prior work has shown that ANS decoders run faster when several independent streams are decoded at once (Giesen on interleaved coders [16]; later work scaling this to many streams [17] and to GPUs [18]). MIC does not claim to invent ANS or interleaving; it applies these known ideas inside a 16-bit-native medical codec with a specific large-alphabet design.

Table I places MIC against the prior work on axes that matter for a web PACS viewer: lossless medical support, 16-bit residual coding, parallel decode, and browser deployability (including JPEG XL, which is ratio-capable but not a feasible 16-bit lossless browser baseline today).

TABLE I

FEATURE COMPARISON OF MIC WITH RELATED CODECS AND ENTROPY CODERS FOR THE VIEWER-RELEVANT AXES. "16-BIT RESIDUAL AS ONE SYMBOL" CONCERNS THE ENTROPY STAGE ONLY: JPEG-LS, HTJ2K, AND JPEG XL HANDLE HIGH-BIT-DEPTH IMAGES, BUT THEIR ENTROPY STAGES DO NOT TREAT THE PREDICTED RESIDUAL AS A SINGLE 16-BIT SYMBOL IN THE SENSE USED HERE. "16-BIT LOSSLESS IN BROWSER" MEANS A PRODUCTION-READY PATH THAT RECONSTRUCTS NATIVE-DEPTH MEDICAL PIXELS CLIENT-SIDE AT THE TIME OF WRITING. SOURCE-LINE COUNTS ARE ORDER-OF-MAGNITUDE CONTEXT.

Feature	JPEG-LS	HTJ2K	JPEG XL	Zstd	rANS	MIC
Lossless medical	Yes	Yes	General	General	General	Yes
16-bit residual as one symbol	No	No	No	No	Configurable	Yes
Large alphabet (symbol cap)	N/A	N/A	N/A	4,096	Configurable	65,535
Adaptive table sizing	No	No	Yes	No	No	Yes
Multi-state ILP	No	No	No	No	Yes	Yes
Strip-parallel decode	No	Yes	Yes	Yes	Yes	Yes
16-bit lossless in browser	Via WASM	Via WASM	No	Partial	No	Purpose-built JS
Source lines	5k	20k	Large	N/A	N/A	1.1k

III. How MIC Works

The pipeline is three stages, each simple enough to decode in one straight-line pass with very few branches: raw 16-bit pixels go through spatial prediction (subtract a guess based on neighbors), then 16-bit run-length encoding (collapse repeated values), then FSE entropy coding (remove the remaining redundancy), producing compressed bytes.

A. Spatial prediction

For each pixel, MIC predicts its value as the **average of the pixel above and the pixel to the left**, then stores only the difference (the residual). Pixels on the top edge or left edge use whatever single neighbor is available; the very first pixel is stored as-is. Division rounds down, and the decoder repeats the exact same arithmetic so reconstruction is exact.

This predictor was chosen because it is inexpensive, has almost no branches on the decode side, and works well on medical images.

a) Other predictors we evaluated: We compared four predictors (left-only, average, Paeth from PNG [20], and MED from JPEG-LS [5]) through the full pipeline. MED gave the best ratio (about 2.4% better than average) but decoded 1.5–2× slower because of its three-way branch and a diagonal dependency that blocks the fast branch-free loop. The average predictor was preferred for its consistency and speed, so it is the default. Table II summarizes the comparison.

TABLE II

PREDICTOR ABLATION SUMMARY: GEOMETRIC MEANS AND WIN COUNTS ACROSS ALL 39 IMAGES.

Predictor	Geo. Mean	Wins	Avg→X
Left-only	3.22×	2/39	−4.4%
Average (MIC)	3.36×	11/39	baseline
Paeth	3.39×	0/39	+0.8%
MED (JPEG-LS)	3.44×	26/39	+2.4%

B. Handling large jumps: overflow coding

Most residuals are small, but occasionally a pixel jumps far from its prediction. Rather than widen the whole stream to 32 bits to handle rare large values, MIC reserves one special code as a **delimiter**: small, in-range residuals are stored directly as 16-bit codes (with zero mapped to a fixed midpoint value); a large, out-of-range residual is stored as the delimiter followed by the raw pixel value.

On decode, the reader checks each value: if it is the delimiter, the next value is a raw pixel; otherwise it is a normal residual. The ranges are designed not to overlap, so there is never any ambiguity. In normal 8–16-bit clinical data the overflow path is rare; it exists to guarantee correctness, not for the common case. Table III shows the mapping for a representative bit depth.

TABLE III
OVERFLOW CODING EXAMPLE FOR $d = 12$ BITS (DELTATHRESHOLD= 2047, DELIMITER $D = 4095$).

Residual r	Emitted code(s)	Notes
−2	2045	in-range
−1	2046	in-range
0	2047	in-range (= threshold)
+1	2048	in-range
+2	2049	in-range
±2047	4095, raw pixel	overflow

C. 16-bit run-length encoding (RLE)

The residual stream is turned into two kinds of runs: **same runs** (a count plus one repeated value, used when a value appears at least three times in a row) and **literal runs** (a count followed by that many values copied verbatim; the values need not be distinct). A single header number tells the two apart using a midpoint trick: small header values mean "same run of this length," large ones mean "literal run." The longest single run is ~32,767 symbols; longer stretches are split across several headers automatically.

Why 16-bit RLE matters. Suppose 1,000 identical 16-bit residuals appear in a row. MIC encodes that as just two numbers: a count and the value. A *byte*-oriented RLE sees the high and low bytes interleaved, so no single byte repeats 1,000 times, so it is forced into two interleaved runs or falls back to literals. Working at 16 bits captures the repetition that byte-splitting hides.

Fast decode. Same runs (often 80% of all runs on smooth images) are fast-pathed: the decoder just hands back the cached value and decrements a counter, without touching the compressed data at all.

D. The entropy coder (FSE) for large alphabets

FSE in one paragraph. FSE keeps a single integer "state." To decode one symbol, it looks up the current state in a table; the entry tells it which symbol to output, how many bits to read next, and how to compute the next state. So each symbol costs **one table lookup, one bit read, and one addition**, with no multiplies or divides. (Encoding runs the symbols in reverse and builds the bitstream backwards; decoding reads forwards. That reversal is inherent to how ANS works.)

MIC extends FSE to handle up to **65,535 symbols** instead of Zstandard's 4,096, because high-bit-depth residuals genuinely require a large alphabet. It also **sizes its tables to the data**: an 8-bit image stored in 16-bit containers (common for MR) shrinks the working tables from ~512 KB down to about 2 KB, which keeps everything in fast cache.

Picking the table size automatically. FSE has a "tableLog" parameter that sets the size of the probability table: the table has $2^{tableLog}$ entries, so each increment of tableLog doubles it. A larger table represents each symbol's probability at finer granularity, which can improve the compression ratio, but it uses more memory and takes longer to build, and on a small or low-entropy image the extra precision is wasted. MIC sets tableLog from two measurements of the post-prediction symbol stream that the entropy stage is about to encode:

- the **alphabet size** (written β): the number of distinct symbol values that actually occur in the stream; and
- the **symbol density** (written C_r): the average amount of data backing each symbol, computed as the total number of symbols divided by the alphabet size β.

MIC uses the larger table (tableLog 12 instead of the default 11) only when *both* quantities are high enough, specifically more than 128 distinct symbols ($\beta > 128$) and more than 32 symbols of data per distinct value ($C_r > 32$). The intuition is that a finer table pays off only when the alphabet is genuinely large *and* each of its symbols is observed often enough to estimate the extra probabilities reliably; a large alphabet with little data per symbol (C_r small) would instead produce noisy frequency estimates and waste header space. On the test set this rule turned on the larger table for exactly the nine images that benefit (1–10% better ratio), with no effect on decode speed. Table IV shows the ablation across selected images.

TABLE IV
TABLELOG ABLATION: FORCED TL=11/12/13 VS. ADAPTIVE SELECTION (SELECTED IMAGES, ARM64 REFERENCE PLATFORM).

Image	TL=11	TL=12	TL=13	Adaptive
CR	3.47×	3.63×	3.69×	3.69×
MG1	8.00×	8.57×	8.79×	8.79×
MG2	7.98×	8.55×	8.77×	8.77×
MR2	3.14×	3.24×	3.28×	3.28×
RG2	3.85×	4.12×	4.23×	4.23×
RG3	5.64×	5.95×	6.08×	6.08×

IV. FASTER DECODING: MULTI-STATE DECODING

A. The bottleneck

A standard single-state ANS decoder has an inherent limitation: each symbol depends on the one before it. To decode symbol N the decoder needs the state produced by symbol $N - 1$, so the work cannot start early. With a table lookup taking roughly 4–5 CPU cycles, the loop is limited to about one symbol every few cycles, even though modern CPUs have 4–6 execution ports that remain largely idle. A single decode chain uses only one of them, leaving ~75–83% of the CPU's capacity unused. That idle capacity is exactly what a viewer

needs to reclaim if client-side decode is to leave room for fetch and display inside a 25–60 FPS budget.

B. The approach

MIC runs **several independent decoders at once** on interleaved positions. With 8 chains, chain 0 handles positions $0, 8, 16, \ldots$; chain 1 handles $1, 9, 17, \ldots$; and so on. The chains share nothing during decoding, each has its own state and its own stream of lookups, so the CPU's out-of-order engine can keep all of them in flight simultaneously.

On the encoder side, the symbols are split into N interleaved groups, each compressed independently, and the resulting streams are concatenated. They all share one decode table, and each chain's starting state is recorded in the header. The decoder just writes each chain's output back to its positions.

C. Measured speedup

In theory, N chains could be up to $N\times$ faster. In practice the speedup is smaller because the chains share some bookkeeping (refilling the bit reader), the unrolled loop puts pressure on the instruction cache, and reading the compressed data has its own bandwidth limit [21]. Measured on the FSE stage alone, **4 states** give +36% on ARM64 and +12% on AMD64 (relative to single state), and **8 states** a further +5% on ARM64 and +1% on AMD64, with **no change to the compressed file**. ARM64 continues to improve as chains double; AMD64 flattens out earlier because its core already extracts substantial parallelism from a single chain. Table V shows the simple latency model against the measured ARM64 speedup.

TABLE V
MULTI-STATE DECODER LATENCY MODEL AND EMPIRICAL SPEEDUP.

States (k)	Amortized latency	Theoretical	Empirical (ARM64)
1	L cy/sym	1.0×	1.0×
2	$L/2$ cy/sym	2.0×	1.3×
4	$L/4$ cy/sym	4.0×	1.5×
8	$L/8$ cy/sym	8.0×	1.6×

The decoder signals its mode with a short magic header (a reserved byte value that cannot be a valid single-state setting), so single-, two-, four-, and eight-state files all coexist without confusion. On AMD64 and ARM64 the inner loop uses each architecture's native variable-shift instructions; other platforms get a pure-Go fallback. Table VI summarises the on-disk layout of a single-frame stream.

TABLE VI
SINGLE-FRAME MIC BITSTREAM LAYOUT. ALL MULTI-BYTE FIELDS ARE LITTLE-ENDIAN. *STATES* k IS THE NUMBER OF INTERLEAVED FSE CHAINS ($k \in \{1, 2, 4, 8\}$).

Field	Size	Meaning
Magic / mode byte	1 B	Single-state: encodes `tableLog` (value 5–17). Multi-state: `0xFF`.
Multi-state tag	1 B	For multi-state streams, one of `0x02`, `0x04`, `0x84` indicating $k = 2$, 4, or 8.
Symbol count	4 B	Number of decoded RLE symbols.
TableLog	1 B	Decode-table size: $2^{tableLog}$ entries (multi-state path only; the single-state path overloads byte 0).
Normalized counts	variable	FSE normalised symbol frequencies, serialised with the standard variable-length encoding.
Per-chain initial states	$k \times 2$ B	Starting state for each FSE chain.
Per-chain stream lengths	$k \times 4$ B	Compressed-stream length per chain (omitted when $k = 1$).
Per-chain compressed streams	variable	Concatenated chain payloads.

V. Evaluation Methodology

Dataset (systems benchmark, not clinical validation). 39 de-identified DICOM images across eleven modalities (CT, MR, CR, X-ray, mammography, digital radiography, nuclear medicine, PET, fluoroscopy, secondary capture, and tomosynthesis), drawn from the public NEMA WG-04 sample sets [26], the NEMA 1997 CD, a public breast-tomosynthesis case [27], and additional grayscale slices from public GDCM and TCIA collections. Table VII lists the images. The corpus is sized and selected for **modality diversity and viewer stress cases** (tiny PET frames through multi-megabyte mammography and radiography), in the spirit of systems image suites such as Kodak's 24-image set and CLIC validation sets of roughly 41–61 images, rather than as a clinical-validation cohort of the scale used in large interchange studies [23]. Per-image and geometric-mean results should be read as evidence on this set; population-level claims require a larger multi-site corpus (Section VII).

TABLE VII
TEST DATASET: 39 CLINICAL DICOM IMAGES SPANNING ELEVEN MODALITIES.

Image	Modality	Dimensions	Raw (MB)
MR	Brain MRI	256×256	0.13
CT	CT scan	512×512	0.50
CR	Comp. radiography	2140×1760	7.18
XR	X-ray	2048×2577	10.1
MG1	Mammography	2457×1996	9.35
MG2	Mammography	2457×1996	9.35
MG3	Mammography	4774×3064	27.3
MG4	Mammography	4096×3328	26.0
CT1	CT scan	512×512	0.50
CT2	CT scan	512×512	0.50
MG-N	Mammography	3064×4664	27.3
MR1	Brain MRI	512×512	0.50
MR2	Brain MRI	1024×1024	2.00
MR3	Brain MRI	512×512	0.50
MR4	Brain MRI	512×512	0.50
NM1	Nuclear med.	256×1024	0.50
RG1	Radiography	1841×1955	6.86
RG2	Radiography	1760×2140	7.18
RG3	Radiography	1760×1760	5.91
SC1	Sec. capture	2048×2487	9.71
XA1	Fluoroscopy	1024×1024	2.00
CT_BRAIN	CT scan	512×512	0.50
CT_ANKLE	CT scan	512×512	0.50
CT_ORT	CT scan	512×512	0.50
CT_CHEST	CT scan	400×512	0.39
MR_HEAD	Brain MRI	256×256	0.13
MR_INTERA	Brain MRI	1024×1024	2.00
DX_HAND	Digital radiography	1480×1410	3.98
DX_CHEST	Digital radiography	1416×1420	3.84
CR_THORAX	Comp. radiography	2076×1876	7.43
PET1	PET	256×256	0.13
PET2	PET	256×256	0.13
CT_LUNG	CT scan	512×512	0.50
CT_PANCREAS	CT scan	512×512	0.50
MR_BRAIN	Brain MRI	320×256	0.16
MR_BREAST	Breast MRI	256×256	0.13
MR_PROSTATE	Prostate MRI	320×320	0.20
PET_PSMA	PET	200×200	0.08
PET_LUNG	PET	200×200	0.08

Baselines. HTJ2K (via OpenJPH [24]), JPEG-LS (via CharLS [25]), and Delta + Zstandard [14] at its maximum level. All were called in-process (no file or subprocess overhead) for a fair speed comparison.

Metrics. Compression ratio (uncompressed ÷ compressed), decode and encode throughput in MB/s of uncompressed pixels, and, for the multi-frame viewer path, interactive frames per second (FPS) under headless Chromium.

Platforms. Native single-frame tables use two reference machines: an ARM64 host (Apple M4 Pro) and an AMD64 host (AWS Intel Xeon 6). Go code used the standard compiler; C components were built with `-O3` (and `-march=native` on AMD64). Each native benchmark ran the full image 10 times (`-benchtime=10x`, i.e., ten iterations, not ten seconds); run-to-run variance stayed under 2% on ARM64 and 5% on AMD64. JavaScript micro-benchmarks (Tables XIII and XIV) ran under Node.js on the same ARM64 host. Multi-frame cine measurements (Table XV) ran under headless Chromium on a separate ARM64 laptop (Apple M2 Max) using the multi-frame harness under `web/` in the accompanying code. Elsewhere in the paper, platforms are referred to only as ARM64 or AMD64.

The "MIC" decoder column. The single "MIC" column in the result tables always refers to the fastest available C decoder configuration on each platform, so that the comparison against the baselines is C-against-C rather than mixing a pure-Go decoder with optimized C libraries. Concretely, this is the eight-state decoder: eight independent FSE chains decoded in parallel by the out-of-order engine (Section IV), which is the fastest of the 1/2/4/8-state configurations on both reference machines. The entropy stage is identical across platforms; what differs is the vector width applied to the two stages that follow it. On AMD64 the run-length expansion and the prediction-reversal (delta-inverse) passes are accelerated with AVX-512, which widens those memory-bound loops to process more elements per instruction; on ARM64 the same passes run with NEON-width or scalar code. This is purely a code-generation difference: both builds read the *identical* compressed bytes and produce bit-identical output, so a file encoded once decodes correctly on either platform, and the only thing the wider vectors change is decode speed, not the bitstream. The same eight-state stream is also what the strip-parallel mode (Section VI) and the JavaScript decoder consume; they reuse this decoder rather than defining a separate format.

Browser decoder (primary deployment target). A ∼20 KB pure-JavaScript ES module (zero npm dependencies) plus a 2.5 MB Go WebAssembly build implement the viewer path of Section I-A. The JS decoder auto-detects 1/2/4/8-state streams. Micro-benchmarks under Node.js on the ARM64 reference host characterize per-image throughput; multi-frame cine measurements under headless Chromium characterize interactive FPS against the 25–60 FPS target (Section VII-A; hosts in Section V).

Multi-frame cine corpus. In addition to the 39 single-frame images, the viewer evaluation uses seven multi-frame studies (cardiac cine MR, XA angiography, nuclear-medicine gated heart, enhanced CT, breast tomosynthesis, and a CT axial series; frame counts in Table XV). Each study is encoded as independent per-frame MIC bitstreams and played back client-side, matching the fetch–decode–display loop of Section I-A.

VI. Results

A. Compression ratio

JPEG-LS produces the smallest files on nearly every image (37 of 39); its context-adaptive predictor is difficult to beat on pure ratio. For a web PACS viewer, ratio is only one axis of the objective: smaller files reduce fetch time, but decode speed determines whether multi-frame scrolling stays interactive. MIC therefore targets the **ratio–speed–deployability balance** required by client-side viewing rather than archival size alone. Across the dataset MIC reaches a **3.36**× geometric-mean ratio: about 88% of JPEG-LS (3.84×) and within about 2% of HTJ2K (3.42×), while decoding faster than both. Mammography compresses best (up to 8.79×) because its smooth tissue produces long near-zero runs, and studies with large uniform backgrounds reach the highest single-image ratios (PET_PSMA at 10.12×, CT_ANKLE at 9.48×). Table VIII gives the per-image ratios for all variants.

B. Encoding speed

MIC encodes faster than the others on most images, on both platforms, because its encoder is a single pass (predict, run-length, table-driven entropy code) while HTJ2K and JPEG-LS do more work per pixel. On **ARM64**, MIC is fastest on 38 of 39 images: roughly 1.0–3.3× HTJ2K, 1.0–7.1× JPEG-LS, and 30–320× Delta + Zstandard (whose max-level LZ77 search accounts for its low throughput); HTJ2K edges ahead only on the tiny PET_LUNG study. On **AMD64**, MIC is fastest on 32 of 39; HTJ2K leads on the two largest mammography images, one radiography image, and four of the smaller new-corpus studies, because the AMD64 baselines are themselves heavily vectorized and per-image table-build overhead weighs more on tiny images. Table IX gives the per-image encoding throughput.

C. Decoding speed

Decoding is where MIC performs best. Its decoder is dominated by the compact FSE loop (one lookup, one bit read, one add) with no hard-to-predict branches. HTJ2K and JPEG-LS both have data-dependent branches that cause CPU mispredictions. Zstandard is the most comparable codec (same predictor, also uses FSE) but works on bytes, so it pays the 2× byte-splitting cost on wide data. On **ARM64**, MIC is fastest on all 39 images: roughly 1.2–2.1× HTJ2K, 1.0–2.0× Zstandard, and 1.1–5.1× JPEG-LS, with the largest margins on the smaller images. On **AMD64**, MIC is fastest on 36 of 39 images; on the three exceptions (CT_BRAIN, MR_INTERA, and the tiny PET_LUNG study) in-process Zstandard is marginally faster. Across its wins MIC runs about 1.0–1.8× HTJ2K and 1.9–7.1× JPEG-LS, and stays within 0.7–1.8× of Zstandard.

MIC therefore delivers both **a better ratio than Zstandard on most images** (34 of 39) and **faster decoding on nearly every image**, which reflects the 16-bit-native design. JPEG-LS retains about a 14% ratio advantage, so the archival

TABLE VIII
LOSSLESS COMPRESSION RATIOS: ALL CODEC VARIANTS, 39 IMAGES. BOLD = BEST RATIO PER ROW.

Image	Raw (MB)	Δ+Zstd-19	MIC	Wavelet	PICS-4	PICS-8	HTJ2K	JPEG-LS
MR	0.13	1.95×	2.35×	2.38×	2.28×	2.21×	2.38×	**2.52**×
CT	0.50	2.05×	2.24×	1.67×	2.15×	1.96×	1.77×	**2.68**×
CR	7.18	3.24×	3.69×	3.81×	3.70×	3.71×	3.77×	**3.96**×
XR	10.1	1.43×	1.74×	**1.76**×	1.75×	1.75×	1.67×	**1.76**×
MG1	9.35	7.20×	8.79×	8.66×	8.84×	8.87×	8.25×	**8.91**×
MG2	9.35	7.21×	8.77×	8.65×	8.83×	8.85×	8.24×	**8.90**×
MG3	27.3	2.09×	2.24×	2.31×	2.31×	2.33×	2.22×	**2.38**×
MG4	26.0	3.10×	3.47×	3.59×	3.58×	3.62×	3.51×	**3.71**×
CT1	0.50	2.47×	2.79×	2.48×	2.54×	2.29×	2.70×	**3.19**×
CT2	0.50	3.24×	3.48×	2.87×	3.11×	2.72×	3.29×	**4.54**×
MG-N	27.3	2.04×	2.24×	2.32×	2.31×	2.34×	2.23×	**2.38**×
MR1	0.50	1.79×	2.09×	2.14×	2.10×	2.08×	2.12×	**2.30**×
MR2	2.00	2.77×	3.28×	3.34×	3.31×	3.31×	3.35×	**3.52**×
MR3	0.50	3.47×	3.92×	4.09×	3.89×	3.83×	4.33×	**4.51**×
MR4	0.50	3.58×	4.12×	4.18×	4.09×	4.03×	4.21×	**4.49**×
NM1	0.50	4.88×	5.15×	5.01×	5.25×	5.28×	5.76×	**6.28**×
RG1	6.86	1.45×	1.70×	1.70×	1.70×	1.69×	1.63×	**1.72**×
RG2	7.18	3.69×	4.23×	4.32×	4.28×	4.30×	4.32×	**4.51**×
RG3	5.91	5.46×	6.08×	6.82×	6.11×	6.12×	6.99×	**7.31**×
SC1	9.71	3.46×	3.71×	3.70×	3.73×	3.73×	3.85×	**4.73**×
XA1	2.00	4.37×	5.01×	4.94×	5.04×	5.03×	4.88×	**5.39**×
CT_BRAIN	0.50	3.08×	3.06×	2.89×	2.77×	2.47×	3.39×	**4.28**×
CT_ANKLE	0.50	9.30×	**9.48**×	5.02×	9.30×	9.14×	4.97×	6.87×
CT_ORT	0.50	2.36×	2.68×	2.38×	2.51×	2.26×	2.56×	**3.00**×
CT_CHEST	0.39	2.46×	2.82×	2.10×	2.52×	2.21×	2.23×	**3.22**×
MR_HEAD	0.13	2.11×	2.61×	2.65×	2.57×	2.53×	2.67×	**2.86**×
MR_INTERA	2.00	4.71×	3.68×	4.05×	3.75×	3.78×	5.04×	**5.08**×
DX_HAND	3.98	1.99×	2.24×	2.35×	2.26×	2.25×	2.37×	**2.48**×
DX_CHEST	3.84	1.43×	1.66×	**1.82**×	1.65×	1.63×	1.63×	1.70×
CR_THORAX	7.43	1.61×	1.82×	1.81×	1.84×	1.83×	1.81×	**1.90**×
PET1	0.13	2.37×	2.74×	2.81×	2.64×	2.52×	3.02×	**3.21**×
PET2	0.13	3.39×	3.39×	3.48×	3.16×	2.58×	4.37×	**4.74**×
CT_LUNG	0.50	2.40×	2.73×	2.45×	2.50×	2.25×	2.67×	**3.15**×
CT_PANCREAS	0.50	2.09×	2.36×	1.76×	2.18×	1.98×	1.85×	**2.70**×
MR_BRAIN	0.16	6.17×	7.27×	6.75×	7.31×	7.24×	7.62×	**8.46**×
MR_BREAST	0.13	3.31×	4.01×	3.94×	4.00×	3.91×	4.10×	**4.48**×
MR_PROSTATE	0.20	2.04×	2.30×	2.34×	2.28×	2.26×	2.49×	**2.65**×
PET_PSMA	0.08	11.88×	10.12×	7.67×	1.20×	1.18×	13.91×	**15.78**×
PET_LUNG	0.08	4.56×	2.97×	2.97×	1.00×	1.03×	5.21×	**5.62**×
Geomean	—	**3.06**×	**3.36**×	**3.21**×	**3.04**×	**2.95**×	**3.42**×	**3.84**×

choice between them is decode speed versus storage cost; for interactive viewing the decisive quantity is whether per-frame decode time leaves room for fetch and display at 25–60 FPS (Section VII-A). Per-image numbers are in Table X, and the entropy stage in isolation is in Table XI.

D. Other MIC variants (for reference)

The main tables use one MIC column, but the codebase includes several variants that exist for different deployment needs rather than to compete on raw speed. The **pure Go** build runs the entire pipeline in Go with no C dependency, at roughly 40–60% of the C decoder's speed; this is useful where C cannot be linked (mobile, WebAssembly, locked-down runtimes). The **1/2/4-state decoders** are the same C decoder with fewer chains; they serve as baselines for the multi-state comparison. **Wavelet-based MIC** replaces the predictor with a reversible 5/3 wavelet transform [19] feeding the same back end; it is competitive on a few images but generally slower on this corpus, since smooth medical images do not reward the wavelet's additional cost. The eight-state C decoder is both the simplest and the fastest configuration on this dataset.

E. Multi-threaded decoding

For large single frames that would otherwise miss the viewer frame-rate target, MIC can also use Parallel Image Compressed Strips (PICS): the image is split into N horizontal strips, each with its own FSE table and independently decodable bitstream, and a C pthreads (or browser worker) pool decompresses the strips concurrently. This scales nearly linearly up to four threads on both platforms and keeps scaling to eight threads on images above ∼1 MB, hitting about **4.3 GB/s** on the largest mammography images on ARM64. Small images do not benefit: thread setup costs more than it saves, and on the two 0.08 MB PET images the strips fall back toward raw storage, so strip parallelism does not apply. We report this separately because the other codecs run single-threaded in our pipeline, so a direct comparison would not be fair. Table XII gives the per-image strip-parallel numbers.

Practical guidance: to decode many small images, use single-threaded MIC per request, which is already fast. To decode one large image on a multi-core machine, use strip-parallel mode with four to eight threads.

F. Client-side decoder for web PACS

The primary deployment path for the viewer workflow is the pure-JavaScript decoder. Notably, the **four-state** variant

TABLE IX
SINGLE-THREADED ENCODING THROUGHPUT (MB/S). MIC IS THE EIGHT-STATE C ENCODER. BOLD MARKS THE FASTEST CODEC PER ROW ON EACH PLATFORM.

	ARM64				AMD64			
Image	MIC	Δ+Zstd	HTJ2K	JPEG-LS	MIC	Δ+Zstd	HTJ2K	JPEG-LS
MR	**457**	7	236	91	**344**	5	246	62
CT	**476**	7	183	126	**280**	5	207	97
CR	**525**	2	186	110	**322**	2	286	75
XR	**661**	3	229	113	**400**	4	256	84
MG1	**1,057**	11	414	277	532	8	**632**	211
MG2	**1,047**	12	407	284	552	8	**629**	209
MG3	**639**	2	211	132	**352**	2	245	104
MG4	**883**	5	342	196	**457**	6	408	151
CT1	**615**	9	243	176	**321**	7	271	125
CT2	**536**	7	204	172	**285**	5	271	124
MG-N	**639**	2	214	140	**358**	2	248	104
MR1	**616**	9	205	114	**367**	6	241	86
MR2	**685**	7	216	118	**409**	4	309	85
MR3	**768**	19	264	167	**441**	12	354	123
MR4	**585**	7	212	163	**367**	5	312	142
NM1	**633**	6	208	162	**340**	5	313	115
RG1	**595**	4	218	84	**396**	5	252	66
RG2	**737**	4	263	151	**425**	3	360	100
RG3	**757**	6	276	183	389	4	**412**	126
SC1	**667**	5	225	203	**416**	4	306	169
XA1	**612**	5	203	133	**356**	4	303	98
CT_BRAIN	**352**	5	168	147	239	4	**253**	112
CT_ANKLE	**955**	15	290	327	**450**	11	358	261
CT_ORT	**644**	10	198	138	**350**	7	269	112
CT_CHEST	**463**	6	177	133	**267**	4	234	100
MR_HEAD	**430**	6	211	106	**288**	4	252	65
MR_INTERA	**415**	4	180	115	**269**	3	268	89
DX_HAND	**587**	4	218	102	**378**	3	261	65
DX_CHEST	**636**	6	209	109	**389**	6	241	77
CR_THORAX	**585**	4	226	91	**397**	4	261	68
PET1	**479**	7	240	129	**295**	5	252	111
PET2	**550**	6	236	145	**269**	4	256	103
CT_LUNG	**586**	10	195	141	**337**	7	258	110
CT_PANCREAS	**454**	7	193	130	**292**	5	240	98
MR_BRAIN	**607**	7	229	191	338	6	**351**	141
MR_BREAST	**522**	5	241	141	**292**	4	290	96
MR_PROSTATE	**465**	8	195	97	**313**	6	250	66
PET_PSMA	**434**	14	423	426	296	10	**377**	364
PET_LUNG	232	6	**245**	235	144	5	**260**	167
Geomean	**582**	**6**	**231**	**148**	**343**	**5**	**293**	**110**

runs 6–21% *faster* than eight-state in JavaScript, even on the same hardware where eight states win in C. The reason is the JavaScript engine (V8): it can keep four independent chains busy but not eight, so the extra chains add overhead without shortening the critical path. The eight-state path still works in JavaScript (one decoder accepts C-encoded files without conversion), but four states is the best configuration for the browser.

Concretely, a 9.35 MB mammography image decodes in 17.1 ms (547 MB/s) using four-state strips across worker threads. A web DICOM viewer can therefore fetch a `.mic` object from storage and decode it client-side without server-side transcoding latency. Tables XIII and XIV report Node.js micro-benchmarks on the ARM64 reference host (Section V); Section VII-A reports multi-frame cine rates under headless Chromium against the 25–60 FPS viewing target.

Practitioner takeaway. For simplicity in a pure browser path, ship the four-state JavaScript decoder (and optional PICS workers). For maximum single-threaded server or gateway throughput, use MIC-4state-C / eight-state C as appropriate on the host. For large frames on multicore clients (tomosynthesis, full-field mammography), use PICS with four to eight strips so the decode stage stays inside the 25–60 FPS budget.

VII. DISCUSSION

A. Meeting the viewer frame-rate target

The introduction fixed the success criterion for this paper as interactive multi-frame viewing in a web PACS: roughly 25–60 FPS of decoded pixels for smooth scrolling and cine, with lossless reconstruction at native bit depth and no server-side transcoding. In-browser cine-playback measurements on the ARM64 browser host of Section V (headless Chromium; seven multi-frame studies spanning cardiac cine MR, XA angiography, nuclear-medicine gated heart, enhanced CT, breast tomosynthesis, and CT axial series; harness under `web/` in the accompanying code) confirm that MIC's client-side path meets that target across modalities. Single-threaded MIC-4state decodes cardiac cine MR at approximately 460 FPS, XA angiography at approximately 140 FPS, and a CT axial series at approximately 86 FPS, all above the 60 FPS ceiling of the target range. The heaviest case, breast tomosynthesis at native frame size (about 9.3 MB per frame, comparable to a full mammography image), drops single-threaded throughput to approximately 17 FPS; the strip-parallel decoder recovers approximately 51 FPS with eight strips, which falls inside the target range. Table XV summarises these results.

TABLE X
SINGLE-THREADED DECOMPRESSION THROUGHPUT (MB/S). MIC IS THE EIGHT-STATE C DECODER (AVX-512 ACCELERATION OF THE RLE AND DELTA INVERSE STAGES ON AMD64). BOLD MARKS THE FASTEST CODEC PER ROW ON EACH PLATFORM.

	ARM64				AMD64			
Image	MIC	Δ+Zstd	HTJ2K	JPEG-LS	MIC	Δ+Zstd	HTJ2K	JPEG-LS
MR	**649**	389	328	137	**614**	360	453	91
CT	**495**	406	299	158	**444**	393	329	110
CR	**580**	527	316	181	**570**	479	445	124
XR	**668**	512	324	131	**626**	509	393	88
MG1	**762**	685	612	481	**916**	596	913	336
MG2	**753**	692	622	492	**942**	599	902	334
MG3	**600**	433	314	175	**547**	362	392	118
MG4	**788**	504	508	246	**686**	469	634	158
CT1	**600**	450	357	224	**535**	432	389	142
CT2	**570**	471	342	212	**522**	458	383	131
MG-N	**600**	408	289	180	**549**	357	398	118
MR1	**734**	365	342	149	**586**	381	392	93
MR2	**700**	445	363	211	**682**	460	486	131
MR3	**839**	523	432	289	**742**	497	520	192
MR4	**692**	466	326	220	**664**	496	433	152
NM1	**778**	499	384	260	**597**	515	485	171
RG1	**469**	383	318	123	**547**	340	363	80
RG2	**652**	510	383	225	**716**	503	544	154
RG3	**685**	642	432	294	**719**	564	641	195
SC1	**699**	511	366	263	**673**	483	457	184
XA1	**662**	501	346	244	**670**	541	501	165
CT_BRAIN	**528**	466	326	190	420	**477**	354	122
CT_ANKLE	**853**	587	452	426	**774**	562	537	282
CT_ORT	**538**	442	348	190	**565**	430	384	124
CT_CHEST	**500**	445	297	178	**446**	428	333	110
MR_HEAD	**595**	372	280	167	**679**	371	406	100
MR_INTERA	**590**	563	304	178	454	**527**	436	133
DX_HAND	**500**	414	316	147	**570**	416	362	101
DX_CHEST	**476**	352	307	121	**547**	349	358	83
CR_THORAX	**552**	403	346	129	**561**	365	365	84
PET1	**732**	416	377	197	**679**	384	389	120
PET2	**656**	515	381	227	**487**	438	389	141
CT_LUNG	**522**	316	348	189	**534**	420	383	122
CT_PANCREAS	**517**	451	316	172	**439**	390	337	110
MR_BRAIN	**866**	567	422	375	**708**	514	641	221
MR_BREAST	**640**	545	350	221	**759**	449	510	156
MR_PROSTATE	**679**	422	353	165	**545**	386	408	103
PET_PSMA	**759**	678	475	684	**747**	655	539	395
PET_LUNG	**502**	498	283	278	365	**508**	363	170
Geomean	**631**	**473**	**359**	**215**	**598**	**452**	**445**	**140**

TABLE XI
FSE DECOMPRESSION THROUGHPUT (MB/S) FOR REPRESENTATIVE MODALITIES, SINGLE-THREAD PURE-GO DECODER. INCREASING THE NUMBER OF INDEPENDENT STATE CHAINS EXPOSES MORE INSTRUCTION-LEVEL PARALLELISM TO THE OUT-OF-ORDER ENGINE. PER-IMAGE ROWS ARE TEN REPRESENTATIVE MODALITIES; BOTH GEOMETRIC MEANS ARE OVER ALL 39 IMAGES.

	ARM64 (MB/s)			AMD64 (MB/s)		
Image	1-st	4-st	8-st	1-st	4-st	8-st
MR	1,066	1,576	1,068	656	1,007	847
CT	1,001	1,549	2,142	972	1,034	1,362
CR	2,941	4,882	6,245	1,456	1,844	1,709
XR	3,597	6,081	6,030	1,745	2,341	2,014
MG1	3,558	4,865	4,885	2,216	1,487	1,975
MG-N	3,949	6,647	6,037	1,940	2,545	2,415
NM1	1,971	2,661	2,570	917	1,346	1,356
RG1	1,768	4,356	5,332	1,239	1,709	1,869
DX_HAND	2,120	3,163	4,816	1,245	1,833	1,520
PET1	1,105	1,252	1,530	575	763	616
Geomean (39)	**1,742**	**2,372**	**2,492**	**1,093**	**1,228**	**1,246**

This is the distinction from archival-ratio-focused codecs such as JPEG-LS: a viewer that must sustain frame rate across a multi-frame study benefits more from bounded, fast per-frame decode (and a browser-deployable implementation) than from a further few percent of file-size reduction at the cost of decode latency. The tomosynthesis case is the one study in this evaluation heavy enough to approach the lower edge of the target range on a single thread, and it is also where strip-parallel decoding provides the clearest viewing benefit.

B. Reading the native results with the viewer in mind

Three observations connect the native tables to that workflow. First, MIC's **ratio advantage over Zstandard** (about 10% in geometric mean, higher on 34 of 39 images) comes mainly from coding 16-bit residuals as single symbols rather than byte pairs; the same choice removes per-byte work that would slow client-side decode. Second, on the largest mammography images on AMD64, **HTJ2K's block coder** can amortize setup cost over many pixels and narrow the speed gap; MIC stays within 0–20% there while remaining simpler to ship in JavaScript. Third, **multi-state and strip-parallel decoding** are what keep heavy frames inside the FPS budget; they cost nothing in compressed size for a fixed configuration and are the difference between single-thread tomosynthesis at ∼17 FPS and strip-parallel at ∼51 FPS.

TABLE XII

MULTI-THREADED DECOMPRESSION THROUGHPUT (MB/S) FOR MIC'S STRIP-PARALLEL MODE. THE "MIC" COLUMN REPRODUCES THE SINGLE-THREADED EIGHT-STATE BASELINE FROM TABLE X EXACTLY; N-STRIP COLUMNS DECODE THE SAME COMPRESSED BYTES IN N PARALLEL THREADS. BOTH ARM64 AND AMD64 COLUMNS COVER ALL 39 IMAGES.

	ARM64				AMD64			
Image	MIC	2-strip	4-strip	8-strip	MIC	2-strip	4-strip	8-strip
MR	649	698	939	1,033	614	281	369	291 †
CT	495	511	987	1,549	444	379	641	736
CR	580	865	1,771	3,227	570	768	1,404	2,236
XR	668	888	1,728	3,197	626	787	1,408	2,252
MG1	762	1,415	2,403	4,342	916	1,330	1,924	3,575
MG2	753	1,369	2,226	4,109	942	1,302	1,698	3,011
MG3	600	934	1,763	3,158	547	868	1,597	2,781
MG4	788	1,263	2,185	4,120	686	1,258	1,957	3,107
CT1	600	779	1,178	1,639	535	477	743	797
CT2	570	773	1,249	1,296	522	475	718	806
MG-N	600	988	1,852	3,569	549	871	1,644	2,913
MR1	734	931	1,375	1,898	586	517	729	827
MR2	700	1,003	1,699	2,750	682	786	1,064	1,476
MR3	839	919	1,381	2,035	742	627	861	853
MR4	692	954	1,301	2,063	664	573	813	877
NM1	778	964	1,410	1,868	597	561	777	771
RG1	469	586	1,068	1,964	547	626	1,113	1,833
RG2	652	1,095	1,887	3,269	716	922	1,523	2,494
RG3	685	1,081	2,027	3,289	719	991	1,596	2,504
SC1	699	1,171	2,005	3,174	673	1,042	1,690	2,598
XA1	662	973	1,692	2,404	670	806	1,170	1,443
CT_BRAIN	528	627	1,025	1,291	420	426	630	681
CT_ANKLE	853	1,082	1,745	1,713	774	654	869	877
CT_ORT	538	679	1,099	1,316	565	485	726	778
CT_CHEST	500	622	1,042	1,354	446	409	633	691
MR_HEAD	595	620	915	880	679	306	417	307
MR_INTERA	590	910	1,575	2,405	454	618	940	1,329
DX_HAND	500	605	1,113	2,051	570	619	1,133	1,697
DX_CHEST	476	566	1,003	1,971	547	592	990	1,712
CR_THORAX	552	588	1,128	1,733	561	624	1,141	1,937
PET1	732	637	854	854	679	289	406	317
PET2	656	669	776	923	487	341	356	320
CT_LUNG	522	689	1,107	1,462	534	468	752	808
CT_PANCREAS	517	506	1,018	1,316	439	390	652	688
MR_BRAIN	866	815	999	1,143	708	435	490	448
MR_BREAST	640	653	920	956	759	288	396	337
MR_PROSTATE	679	742	1,093	1,279	545	321	507	455
PET_PSMA‡	759	727	–	–	747	282	–	–
PET_LUNG‡	502	359	–	–	365	208	–	–

†The 130 KB MR image is too small to benefit from 8-way parallelism on AMD64: synchronization overhead exceeds the per-strip work, so 4-strip is faster than 8-strip. ‡On the two 0.08 MB PET images, splitting into four or more strips drives the strip-compressed size to or above the single-stream size, so their 4- and 8-strip figures reflect near-raw copying rather than entropy decoding and are omitted.

TABLE XIII

JAVASCRIPT DECODER THROUGHPUT ON THE ARM64 REFERENCE HOST (NODE.JS V22.16; SECTION V), FOR THE FOUR-STATE AND EIGHT-STATE FSE VARIANTS. MEDIAN OVER 20 ITERATIONS AFTER THREE WARM-UP RUNS.

			4-state		8-state	
Image	Dimensions	Ratio	ms	MB/s	ms	MB/s
MR †	256×256	2.35×	2.9	43	2.7	46
CT	512×512	2.24×	12.3	41	13.1	38
CR	2140×1760	3.69×	137.2	52	165.5	43
MG1	2457×1996	8.79×	70.4	133	80.0	117
MG3	4774×3064	2.29×	560.8	50	607.1	46
DX_HAND	1480×1410	2.24×	82.6	48	96.1	41
PET1	256×256	2.74×	2.1	60	2.5	50

†MR's 0.13 MB image decodes in under 3 ms, at the JavaScript timing floor; the four-state advantage seen on the larger images is within run-to-run noise here.

TABLE XIV

PICS PARALLEL JAVASCRIPT DECODER (`WORKER_THREADS`) ON THE ARM64 REFERENCE HOST (SECTION V), WITH FOUR-STATE AND EIGHT-STATE FSE PER STRIP. WORKER COUNT WAS SWEPT OVER $\{1, 2, 4, 8, 14\}$; THE BEST WALL-TIME FOR EACH VARIANT IS REPORTED.

		4-state strips		8-state strips	
Image	Strips	ms	MB/s	ms	MB/s
MR	4	0.9	134	0.9	135
CT	4	4.1	123	4.5	112
CR	8	22.9	313	28.0	257
MG1	8	17.1	547	16.7	559
DX_HAND	8	17.1	233	17.8	224

TABLE XV

IN-BROWSER CINE-PLAYBACK THROUGHPUT ON THE ARM64 BROWSER HOST (HEADLESS CHROMIUM; SECTION V): MIC-4STATE SINGLE-THREADED VS. STRIP-PARALLEL DECODE.

Study	MIC-4state	MIC-PICS
Cardiac cine MR (16 frames)	~460 FPS	~650 FPS (4-strip)
XA angiography (12 frames)	~140 FPS	~290 FPS (8-strip)
Enhanced CT (2 frames)	~210 FPS	~440 FPS (4-strip)
CT axial series (16 frames)	~86 FPS	~218 FPS (8-strip)
Breast tomosynthesis (16 frames)	~17 FPS	~51 FPS (8-strip)

C. Choosing a codec for viewing versus archiving

For **low-latency web PACS viewing** (series scrubbing, thumbnails, tomosynthesis playback), single-threaded MIC is a sensible default (~600–900 MB/s per request on the native C path), with strip-parallel MIC for one large frame at a time

and the JavaScript decoder when the client is the browser. For **smallest archival files**, JPEG-LS keeps a ∼14% better ratio, at the cost of decoding 1.6–5.7× slower on ARM64. Where **DICOM transfer-syntax compatibility** is required, HTJ2K is the appropriate choice, since it is a standard and is particularly strong on large mammography images on AMD64. Where **no native code is permitted** (browsers, sandboxes), the pure-Go reference or the ∼20 KB JavaScript decoder provides a client-side path without a server transcoder.

A more structured comparison, combining a Pareto view of ratio versus decode speed [28], [29] with a weighted decision matrix over ratio, speed, platform coverage, and deployment ease [30], [31] following deployment taxonomies for DICOM compression [22], [32], puts MIC, HTJ2K, and JPEG-LS on the efficiency frontier (MIC for throughput, JPEG-LS for ratio, HTJ2K in between), with Delta + Zstandard dominated by MIC on both axes. HTJ2K remains the choice whenever standard compatibility is a hard requirement that the scoring does not capture. Table XVI reports the weighted matrix.

TABLE XVI

CODEC SELECTION MATRIX. EACH CRITERION IS NORMALISED SO THE BEST CODEC ON THAT ROW SCORES 1.00. QUANTITATIVE ROWS ARE GEOMETRIC MEANS ACROSS THE 39-IMAGE SET. BROWSER COMPATIBILITY: SHIPPED PURPOSE-BUILT JS OR WASM DECODER USED IN PRODUCTION = 1.00; WASM PORT OF THE NATIVE LIBRARY USED IN CLINICAL VIEWERS = 0.85; NO BROWSER PATH = 0.0. DEPLOYMENT EASE: ZERO EXTERNAL RUNTIME DEPENDENCIES = 1.00; UBIQUITOUS SYSTEM DEPENDENCY = 0.90; C/C++ LIBRARY REQUIRING CMAKE BUILD AND LINK INTEGRATION = 0.70. WEIGHT COLUMNS w_P, w_A, w_B REFLECT TYPICAL PRIORITIES FOR (P) PACS VIEWER / LOW-LATENCY DECODE, (A) ARCHIVAL STORAGE, (B) BROWSER DELIVERY.

Criterion	w_P	w_A	w_B	MIC	HTJ2K	JPEG-LS
Compression ratio	0.20	0.50	0.10	0.88	0.89	**1.00**
Decompression throughput (ARM64)	0.30	0.10	0.15	**1.00**	0.57	0.34
Encoding throughput (ARM64)	0.05	0.10	0.05	**1.00**	0.40	0.25
AMD64 native	0.10	0.05	0.05	**1.00**	**1.00**	**1.00**
ARM64 native	0.10	0.05	0.05	**1.00**	**1.00**	**1.00**
Browser compatibility	0.05	0.00	0.40	**1.00**	0.85	0.85
Deployment ease	0.20	0.20	0.20	**1.00**	0.70	0.70
Weighted score (P) viewer	—	—	—	**0.98**	0.75	0.70
Weighted score (A) archival	—	—	—	**0.94**	0.78	0.80
Weighted score (B) browser	—	—	—	**0.99**	0.77	0.74

D. Future Work

Larger corpora and statistical rigor. The evaluation rests on 39 images across eleven modalities. That is appropriate as a systems throughput suite (diverse enough to exercise smooth mammography, noisy CT, tiny PET, and multi-megabyte radiography), but it is far smaller than clinical-validation studies that may use thousands of images [23], and it is not a controlled multi-site archive sample. Several modalities appear in only two or three images, so per-modality means carry little statistical weight, and geometric means can move with a single outlier. Acquisition conditions absent here (other reconstruction kernels, vendors, bit depths outside the 10–16-bit range tested, or heavy noise) remain uncharacterized. Expanding the corpus and reporting per-modality breakdowns with uncertainty estimates is the priority for treating ratio and FPS margins as general rather than suite-level evidence.

Held-out parameter tuning. The adaptive table-size thresholds ($\beta > 128$, $C_r > 32$) and the choice of the average predictor were informed by this same dataset, so there is a risk that they are fitted to it rather than to medical images in general. Two factors limit, but do not eliminate, that risk: the rules are deliberately coarse (two integer thresholds and a single fixed predictor, with no per-image search), and the thresholds gate a modest 1–10% ratio gain on a minority of images rather than driving the headline result. Even so, a properly held-out study, with the thresholds frozen on one collection and measured on a disjoint one, is required to show they generalize.

Stronger baselines and browser-feasible alternatives. The quantitative comparison covers HTJ2K, JPEG-LS, and Delta + Zstandard as deployable medical or general-purpose baselines. JPEG XL is discussed in Section II: informal in-process tests show a modest ratio advantage for JPEG XL and a marked encode/decode speed advantage for MIC, but JPEG XL is not treated as a viewer baseline because no current browser path decodes it losslessly at 16-bit depth. A full in-process ratio/speed table on the reference platforms remains useful future work for archival comparisons even if it does not change the browser-feasibility argument. Learned lossless codecs (L3C, SReC, and related models) can improve ratio further but remain impractical for real-time web viewing due to runtime dependencies and slow decode; they are excluded for the same reason. Additional isolating baselines (byte-shuffle before Zstandard; basic DICOM RLE) would separate 16-bit-native entropy gains from prediction alone.

E. Adding MIC as a DICOM Transfer Syntax

The evaluation so far operates on the extracted pixel array and on browser-side `.mic` blobs suitable for object-storage delivery to a web viewer. To route the same bitstream through a full PACS archive without a private side-channel, MIC has to be carried inside a DICOM object that existing systems can store and display, which means defining it as a DICOM transfer syntax. We sketch the path here.

Encapsulation. A transfer syntax specifies how the pixel data is serialised into the Pixel Data element (7FE0,0010). MIC fits the existing *encapsulated* pixel-data model that JPEG 2000 and JPEG-LS already use: the element is encoded with undefined length as a sequence of fragments preceded by a Basic Offset Table, and each frame's MIC bitstream is stored as one (or more) fragment items. Because MIC already produces a self-contained per-image bitstream with its own header (Table VI), and a multi-frame container for tomosynthesis, the mapping to fragments is direct and does not require changing the codec.

Identification and attributes. The syntax needs a registered Transfer Syntax UID under the DICOM root, and the associated image-pixel attributes (Bits Allocated, Bits Stored, High Bit, Pixel Representation, Photometric Interpretation, Planar Configuration) must be defined so that a receiver can reconstruct the array unambiguously. Because MIC is lossless and operates on the native sample values, these attributes carry over from the source object without the value re-scaling that lossy syntaxes require.

Standardisation path and an interim option. Formal adoption would proceed through a DICOM Change Proposal:

a public specification of the bitstream, the encapsulation rules, and conformance test vectors, reviewed by the relevant DICOM Working Group. In the interim, two of the partners on a link can negotiate MIC as a privately agreed transfer syntax (a private UID) or carry the bitstream in a private data element, which allows deployment inside a controlled environment before standardisation completes. A reference encoder and decoder, plus a published set of conformance images, would be the practical prerequisites for either route, and are the natural next deliverable beyond the open-source codec.

VIII. Conclusion

Web-based PACS viewing imposes a different objective than archival storage: lossless reconstruction at native 10–16-bit depth with client-side decode fast enough to sustain interactive multi-frame rates (about 25–60 FPS). MIC is a 16-bit-native lossless pipeline designed for that constraint: spatial prediction, 16-bit run-length coding, large-alphabet FSE, multi-state decode, and optional strip parallelism, shipped as a ∼20 KB JavaScript decoder for the browser. On a 39-image, eleven-modality systems-benchmark corpus, MIC reaches a geometric-mean ratio of 3.36× (within ∼2% of HTJ2K and ∼88% of JPEG-LS; about 10% smaller than Delta + Zstandard on 34 of 39 images), is the fastest of four in-process decoders on all 39 images on ARM64 and 36 of 39 on AMD64, and meets the cine frame-rate target in-browser, including strip-parallel recovery on tomosynthesis-sized frames. The individual stages are classical; the contribution is the combination that jointly delivers native 16-bit coding, high decode throughput, and browser deployability for interactive DICOM review. Future work includes larger multi-hospital corpora, held-out parameter validation, a full in-process JPEG XL ratio/speed table on the reference platforms, a DICOM transfer-syntax path, and other archival baselines that remain infeasible for 16-bit lossless browser decode at the time of writing. MIC is open source: https://github.com/pappuks/medical-image-codec.

References

[1] NEMA, "Digital Imaging and Communications in Medicine (DICOM)," PS3.5—Data Structures and Encoding, https://www.dicomstandard.org/, 2024.

[2] D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Springer, 2002.

[3] D. S. Taubman, "High throughput JPEG 2000 (HTJ2K): Algorithm, performance evaluation, and potential," *Proc. SPIE*, vol. 11137, 2019.

[4] D. Taubman *et al.*, "High Throughput JPEG 2000 for Video Content Production and Delivery Over IP Networks," *Front. Signal Process.*, vol. 2, Article 885644, Apr. 2022.

[5] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, 2000.

[6] X. Wu and N. Memon, "Context-based, adaptive, lossless image coding," *IEEE Trans. Commun.*, vol. 45, no. 4, pp. 437–444, 1997.

[7] J. Sneyers and P. Wuille, "FLIF: Free Lossless Image Format based on MANIAC Compression," *Proc. IEEE ICIP*, 2016.

[8] J. Alakuijala *et al.*, "JPEG XL next-generation image compression architecture and coding tools," *Proc. SPIE 11137*, Sep. 2019.

[9] F. Mentzer, E. Agustsson, M. Tschannen, R. Timofte, and L. Van Gool, "Practical Full Resolution Learned Lossless Image Compression," *Proc. IEEE/CVF CVPR*, pp. 10629–10638, 2019.

[10] S. Cao, C.-Y. Wu, and P. Krähenbühl, "Lossless Image Compression through Super-Resolution," arXiv:2004.02872, 2020.

[11] J. Duda, "Asymmetric numeral systems," arXiv:0902.0271, 2009.

[12] J. Duda, "Asymmetric numeral systems: entropy coding combining speed of Huffman coding with compression rate of arithmetic coding," arXiv:1311.2540, 2013.

[13] Y. Collet, "Finite State Entropy—A new breed of entropy coder," https://github.com/Cyan4973/FiniteStateEntropy, 2013.

[14] Y. Collet and M. Kucherawy, "Zstandard Compression and the 'application/zstd' Media Type," RFC 8878, IETF, Feb. 2021.

[15] S. Mahapatra and K. Singh, "An FPGA-based implementation of multi-alphabet arithmetic coding," *IEEE Trans. Circuits Syst. I*, vol. 54, no. 8, pp. 1678–1686, 2007.

[16] F. Giesen, "Interleaved entropy coders," arXiv:1402.3392, 2014.

[17] F. Lin, K. Arunruangsirilert, H. Sun, and J. Katto, "Recoil: Parallel rANS Decoding with Decoder-Adaptive Scalability," *Proc. 52nd ICPP '23*, pp. 31–40, 2023.

[18] A. Weissenberger and B. Schmidt, "Massively Parallel ANS Decoding on GPUs," *Proc. 48th ICPP '19*, Article 100, 2019.

[19] D. Le Gall and A. Tabatabai, "Sub-band coding of digital images using symmetric short kernel filters and arithmetic coding techniques," *Proc. IEEE ICASSP*, pp. 761–764, 1988.

[20] W3C, "Portable Network Graphics (PNG) Specification (Second Edition)," ISO/IEC 15948:2003, Nov. 2003.

[21] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[22] F. Liu *et al.*, "The Current Role of Image Compression Standards in Medical Imaging," *Information*, vol. 8, no. 4, p. 131, Oct. 2017.

[23] D. A. Clunie, "Lossless Compression of Grayscale Medical Images: Effectiveness of Traditional and State-of-the-Art Approaches," *Proc. SPIE Medical Imaging*, 2000.

[24] A. N. Aous, "OpenJPH: An open-source implementation of High-Throughput JPEG 2000," https://github.com/aous72/OpenJPH, 2024.

[25] Team CharLS, "CharLS: A C/C++ JPEG-LS library implementation," https://github.com/team-charls/charls, 2024.

[26] NEMA Working Group 4 (WG-04), "DICOM Compression Sample Images," National Electrical Manufacturers Association, https://www.dicomstandard.org/Resources/Open-Content/Compression-Samples, 2024.

[27] D. A. Clunie, "Breast Tomosynthesis Case 1," DICOM sample images, http://www.dclunie.com/pixelmedimagecases/tomo/case1/, 2009.

[28] R. Geldreich, "Quick survey of the lossless decompression Pareto frontier," 2015, http://richg42.blogspot.com/2015/11/the-lossless-decompression-pareto.html.

[29] J. Sneyers, "JPEG XL and the Pareto front," Cloudinary Blog, 2022, https://cloudinary.com/blog/jpeg-xl-and-the-pareto-front.

[30] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*. New York, NY, USA: McGraw-Hill, 1980.

[31] K. Sayood, *Introduction to Data Compression*, 5th ed. Burlington, MA, USA: Morgan Kaufmann, 2017.

[32] D. A. Clunie, "DICOM support for compression: More than JPEG," *Medical Imaging Informatics and Teleradiology (MIIT)*, 2009, https://www.dclunie.com/papers/MIIT_2009_DicomCompression_Clunie.pdf.